

Unit 1.3

Asynchronous Node.js

Callbacks, promises and async/await — three ways to write the same program

the whole session

```
findUser(n, (e, u) => {  
  findOrders(u, (e, o) => {  
    // ...  
  });  
});  
  
const u = await findUser(n);  
const o = await findOrders(u);
```

01

Callbacks

Where asynchronous JavaScript started, and why you still need to read it

Why a callback exists at all

- A function that waits for the disk cannot return its result: at the moment it returns, the result does not exist yet.
- So instead of returning a value, it accepts a function — and calls it later, when the value arrives. That function is the callback.
- Nothing magical is happening. The callback is an ordinary function passed as an argument; the runtime keeps a reference to it and invokes it from the event loop.
- This is why the code after an asynchronous call runs before the callback: the call only registered interest, it did not wait.

00-callback.js

```
op((result) => {  
  console.log('4 -', result);  
});  
console.log('2 - waiting');  
  
// 1 - starting  
// 2 - waiting  
// 3 - finished  
// 4 - the result
```

code `async/00-callback.js`

Read it out loud "Do this, and when you are done, run that." Everything else in this session is a nicer way of saying the same sentence.

The error-first convention

Node.js callbacks take the error first and the value second. There is nothing to try/catch.

async/01-callback.js

```
function fetchTemperature(sensorId, callback) {
  setTimeout(() => {
    if (!Number.isInteger(sensorId) || sensorId <= 0) {
      return callback(new Error('invalid id'));
    }
    callback(null, { sensorId, celsius: 21.4 });
  }, 500);
}

fetchTemperature(id, (err, reading) => {
  if (err) { console.error(err.message); return; }
  console.log(reading.celsius, '°C');
});
```

- Pass an Error object, never a string — you lose the stack trace otherwise.
- Always return after handling the error, or the success path runs too.
- An asynchronous error is not thrown: a try/catch around the call catches nothing.
- Forgetting to check err is the most common bug in this style.

Callback hell is not about ugliness

Four steps where each one needs the previous result. Nothing here is written badly.

async/02-callbackhell.js

```
findUser('Alice', (err, user) => {  
  if (err) return console.error(err);  
  
  findOrdersOf(user, (err, orders) => {  
    if (err) return console.error(err);  
  
    findInvoiceOf(orders[0], (err, invoice) => {  
      if (err) return console.error(err);  
  
      markAsPaid(invoice, (err, paid) => {  
        if (err) return console.error(err);  
        console.log('done:', paid);  
      });  
    });  
  });  
});
```

The real costs

- You read it inside out, not top to bottom.
- The error check is duplicated at every level, and one omission breaks it silently.
- There is no way to say "do these two at the same time".
- Extracting a step into its own function means rewriting the nesting.

02

Promises

An object that stands for a value you do not have yet

Three states, one transition

A promise starts pending and settles exactly once. After that it never changes again.

pending

The work is still running. This is what you get back the instant you call the function.

fulfilled

It produced a value. `resolve(value)` was called; `.then()` receives that value.

rejected

It produced an error. `reject(error)` was called, or the executor threw; `.catch()` receives it.

- That immutability is the whole point: a settled promise is a safe value you can pass around, store, and attach handlers to as many times as you like.
- You will rarely write `new Promise()` yourself — the standard library already returns promises (`node:fs/promises`, `fetch`), and `util.promisify` converts callback APIs.

code `async/03-promises.js`

Chaining: the same four steps

Whatever you return from a `.then()` becomes the input of the next one.

async/04-promisechaining.js

```
findUser('Alice')
  .then((user) => findOrdersOf(user))
  .then((orders) => findInvoiceOf(orders[0]))
  .then((invoice) => markAsPaid(invoice))
  .then((paid) => console.log('done:', paid))
  .catch((err) => {
    console.error('the sequence failed:', err.message);
  });
```

- The code grows downwards, and you read it in the order it happens.
- One `.catch()` covers every step above it.
- Return a promise and the chain waits for it. Forget the return and the next step gets undefined — the classic bug.

Still not perfect Every step is a separate function scope, so a value from step 1 is not in scope in step 4. That is what `async/await` fixes.

How errors travel

A rejection skips every `.then()` until it finds a `.catch()`. What the catch returns decides what happens next.

```
async/05-promisehandleerror.js

// 1. the rejection skips the middle steps
fail('database unreachable').then(() => /* never runs */).catch(log);

// 2. returning a value RECOVERS the chain
fail('cache miss').catch(() => ({ source: 'fallback' })).then(carryOn);

// 3. rethrowing propagates it – keep the original as `cause`
fail('ECONNREFUSED').catch((err) => {
  throw new Error('could not load the report', { cause: err });
});
```

The one that bites A rejected promise with no `.catch()` anywhere terminates the process. `process.on('unhandledRejection')` is for reporting, never for control flow.

Callbacks and promises, side by side

Callbacks

- Nesting grows with each dependent step.
- Error handling repeated at every level.
- No built-in way to run several in parallel.
- Can be called twice, or never — nothing enforces it.

Promises

- A flat chain read from top to bottom.
- One `.catch()` for the whole sequence.
- `Promise.all` and friends for concurrency.
- Settles exactly once, guaranteed by the specification.

But callbacks are not legacy. Event emitters, streams and most of the Node.js core API still use them. You need to read both fluently.

03

async / await

The same promises, written as if nothing were asynchronous

The version you will actually write

Three rules: an async function returns a promise, await pauses that function, errors are exceptions again.

async/06-asyncawait.js

```
async function settleInvoiceOf(name) {  
  const user    = await findUser(name);  
  const orders  = await findOrdersOf(user);  
  const invoice = await findInvoiceOf(orders[0]);  
  return markAsPaid(invoice);  
}  
  
try {  
  const paid = await settleInvoiceOf('Alice');  
} catch (err) {  
  console.error(err.message);  
}
```

- user is still in scope four lines later — that is what chaining could not give you.
- await pauses this function, not the thread. The event loop keeps serving everyone else.
- Nothing new happens at runtime: this compiles down to the same promises.

Top-level await works in ES modules, so a small script needs no main() wrapper. In CommonJS you still need one.

The mistake worth making once

Two requests that do not depend on each other, awaited one after the other. It works, and it is twice as slow.

```
sequential — 600 ms
```

```
const a = await delay(300);  
const b = await delay(300);
```

```
// b did not need a.  
// We waited for nothing.
```

```
concurrent — 300 ms
```

```
const [a, b] = await Promise.all([  
  delay(300),  
  delay(300),  
]);
```

```
// Start both, then wait.
```

Pick the right combinator

Promise.all all must succeed; rejects on the first failure

Promise.allSettled never rejects; reports the outcome of each one

Promise.race first to settle wins — the idiom for a timeout guard

Promise.any first to SUCCEED wins; rejects only if all of them fail

code [async/10-combinators.js](#)

04

Asynchronous work in practice

Files, HTTP and a database — the three things every API does

Files: three flavours of the same call

node:fs offers every operation in callback, synchronous and promise form. Use the last one.

async/07-files.js

```
import fs from 'node:fs/promises';

await fs.mkdir(dir, { recursive: true });
await fs.writeFile(file, JSON.stringify(data), 'utf8');
const raw = await fs.readFile(file, 'utf8');

try {
  await fs.readFile(missing, 'utf8');
} catch (err) {
  if (err.code === 'ENOENT') handleMissing();
  else throw err; // never swallow the unexpected
}
```

- Without 'utf8' you get a Buffer, not a string.
- Check err.code, not the message: ENOENT, EACCES, EEXIST are stable, wording is not.
- readFileSync is fine in a script that does one thing and exits. In a server it is a bug.
- Use node:path to build paths — never concatenate with slashes.

HTTP: fetch is built in

No axios, no node-fetch, and definitely not request-promise-native — deprecated for years.

async/08-apirequest.js

```
async function getJson(url) {  
  const response = await fetch(url, {  
    headers: { Accept: 'application/json' },  
    signal: AbortSignal.timeout(5000),  
  });  
  
  if (!response.ok) { // 404 and 500 land here  
    throw new Error(`${response.status} ${response.statusText}`);  
  }  
  return response.json();  
}
```

Two traps

- fetch does NOT reject on 404 or 500. Only a network failure rejects. You must check response.ok yourself.
- A request with no signal can hang forever. AbortSignal.timeout is one line and saves you an outage.

Preview of Unit 2 You are about to write the other side of this call — every status code and header here is one you will be choosing.

SQLite, with nothing installed

node:sqlite has been part of Node.js since v22.5. The API is synchronous on purpose.

```
async/09-dboptions.js
```

```
import { DatabaseSync } from 'node:sqlite';
const db = new DatabaseSync('task-manager.db');

db.exec(`CREATE TABLE IF NOT EXISTS users (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL UNIQUE)`);

const insert = db.prepare('INSERT INTO users (name) VALUES (?)');
const alice = insert.run('Alice'); // .lastInsertRowid

db.prepare('SELECT * FROM users WHERE id = ?').get(alice.lastInsertRowid);
db.prepare('SELECT * FROM users').all();
```

- Placeholders (?) always. Concatenating SQL strings is how injection happens.
- Synchronous is correct here: a local query is microseconds of CPU, not a wait. A remote database is I/O and must be awaited.
- Still prints an ExperimentalWarning on Node 24; the npm packages sqlite + sqlite3 are the promise-based alternative.

Habits that save you hours

Writing it

- `async/await` by default; promises when you need a combinator; callbacks only when an API forces you.
- Never mix styles in one function. An async function that also takes a callback is a function nobody can call correctly.
- Await only what you depend on. Independent work goes into `Promise.all`.
- Wrap errors with `{ cause }` instead of replacing them.

Debugging it

- `node --inspect-brk file.js`, then open `chrome://inspect` or attach your editor. Breakpoints work across `await`.
- An async stack trace that ends at the event loop means you forgot to `await` something.
- `console.log` is fine while learning; a structured logger is what you ship.
- `node --test` runs your test files with no dependency to install.

The error you will see most "Cannot read properties of undefined" one line after an async call almost always means a missing `await`: you are reading a property of a `Promise`.

KEY TAKEAWAYS

What to keep from this session

Three styles, one mechanism.

Callbacks, promises and async/await all end up as callbacks queued on the event loop.

await pauses the function, not the thread.

And awaiting independent work in sequence is the most common performance bug there is.

Handle the error where you can do something about it.

try/catch with async/await, one .catch() per chain, err.code for I/O, never an unhandled rejection.

Check what the standard library already gives you.

fetch, fs/promises, sqlite, a test runner and a watch mode are all built in.